3 File Systems and Their Commands

1 Introduction to File Systems

In the Linux operating system, "everything is either a file or a process." This philosophy goes beyond traditional text files, images, and programs—it also includes disk partitions, hardware device drivers, and directories.

> 📖 DEFINITION 1: **File System**
>
> A **file system** is the component responsible for controlling how data is stored, organized, and retrieved on a storage device.
>
> Without a file system, all the information stored on a hard drive or SSD would just be an indistinguishable, single block of raw data, making it impossible to tell where individual files begin and end.

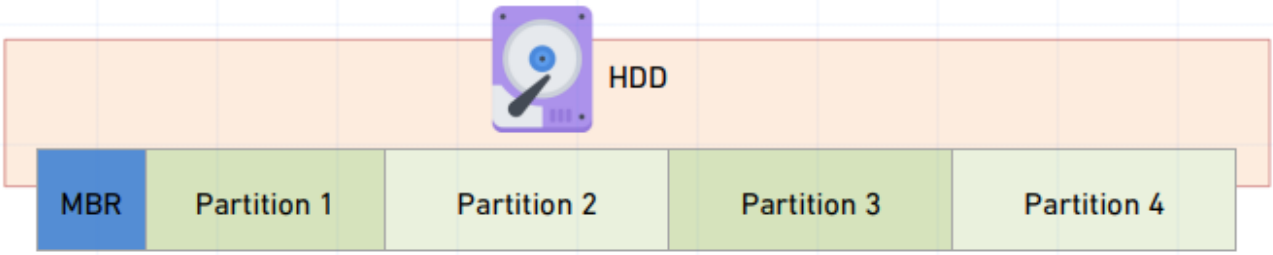
1.1 Key Functions of a File System

- **Data Organization:** It structures data into logical units called **files** and **directories (folders)**, arranging them in a hierarchical tree structure.
- **Metadata Management:** It keeps track of essential attributes for every file, such as its name, size, creation/modification timestamps, access permissions, and ownership.
- **Storage Allocation:** It maps out where files are physically located across the storage blocks of a disk partition, ensuring space is allocated efficiently.

1.1.1 Disk Partitioning

> 📖 DEFINITION 2: **Disk Partitioning**

> A raw storage device initially appears as empty, raw space. Through partitioning, it is logically divided into independent segments called **Partitions**.



Partitioning can be performed using standards such as:

- **MBR (Master Boot Record):** Allocates a small space at the beginning of the drive to hold startup metadata and the partition table.*(Crucial Note: If the partition table is lost, all data is lost because it maps the start and end points of each partition.)*
- **GPT (GUID Partition Table).**

Every partition is managed individually by a **File System** (such as FAT32, NTFS, ext2, ext3, ext4, or XFS). Without a file system, stored information would be an unidentifiable, single piece of data.

2 Files in Linux

In the Linux operating system, **"everything is a file; if it is not a file, it is a process."** The term "file" goes beyond traditional text documents, images, and programs—it also encompasses partitions, hardware device drivers, and directories.

2.1 File Characteristics & Attributes

The file system provides every file with a set of essential properties and information, including:

- **File-Name:** The name given to a file to identify and access it within the system. It is defined by the user or the system and must be unique within its parent directory.

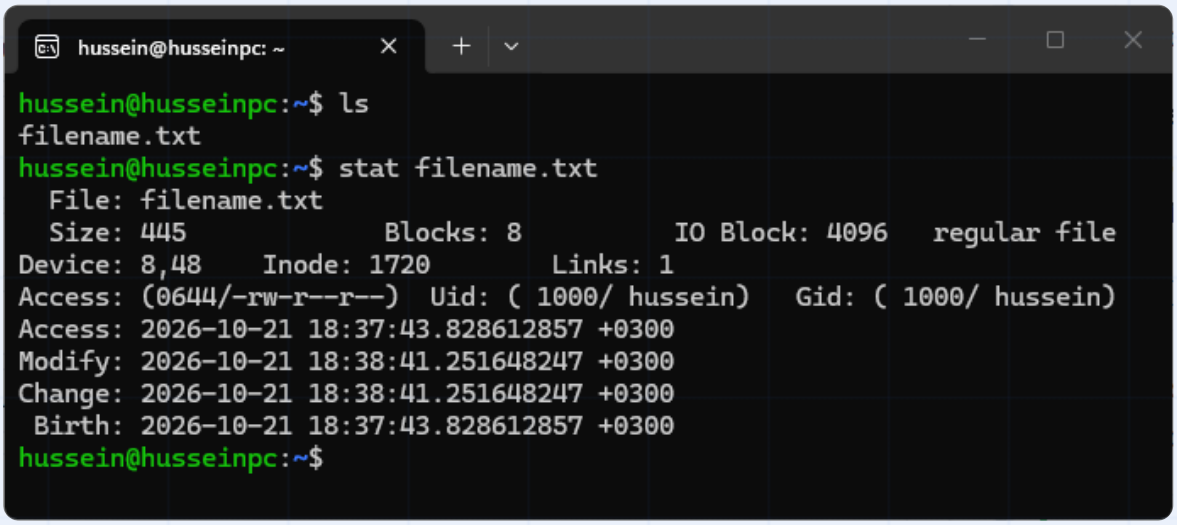
- **Metadata:** Descriptive data about the file, such as its size, creation date, last modification timestamp, access permissions, owner, and associated user group. Metadata assists in managing files and determining access rules.
- **Inode (Unique Number):** A unique identifier assigned to every file in the file system. The Inode contains file metadata and points to the physical location of the file's data blocks on the disk. The system uses the Inode for internal operations instead of the file name.

📝 EXAMPLE 1: Illustrating File Metadata & Inodes (stat)

The `stat` command is the best tool to inspect a file's detailed attributes, including its **Inode number**,**Metadata** (such as size, timestamps, and access permissions), and block information.

```bash
stat filename.txt
```

✅ Show Solution



Breakdown of the Output:

1. **File Type & Size:** Indicates it is a `regular file` with a size of `445` bytes occupying `8` blocks on the disk.

2. **Inode Number (`Inode: 1720`):** The unique system index number that the Linux kernel uses internally to reference this file instead of its human-readable name.
3. **Permissions & Ownership (`Access: (0644/ ...`, `Uid/Gid`):** Shows that the owner `hussein` (UID 1000) has read/write permissions, while members of the group and others have read-only access.
4. **Timestamps:**
   - **Access:** The last time the file was read.
   - **Modify:** The last time the file's content was altered.
   - **Change:** The last time the file's metadata or attributes were modified.
   - **Birth:** The exact creation timestamp of the file.

2.2 File Types

File types indicate the operating system's ability to distinguish between different categories of data. Linux supports several file types:

1. Regular Files (`-`)

- Also known as **General Files**, these are standard packages of information with a specific name and extension.
- They are used to store data such as text documents, code, and images. Users can perform standard operations like adding, modifying, deleting, or removing these files entirely. They are represented by a dash (`-`).

2. Directory Files (`d`)

- Directories (folders) act as containers that hold other files and subdirectories.
- They are organized in a hierarchical, tree-like structure, meaning a directory can contain other directories within it. They are represented by the letter (`d`).

3. Special / Device Files

- These are files used for specialized purposes, often referred to as **Device Files**.
- Unlike operating systems like Windows—which represent hardware such as hard drives and CD-ROMs with drive letters like `C:` or `D:` —Linux represents hardware devices as special files within the file system:
  - **Block Files (`b`):** Represent block storage devices (like hard drives and SSDs) that transfer data in fixed blocks.
  - **Character Device Files (`c`):** Represent devices that transfer data as a stream of characters (like keyboards and mice).
  - **Named Pipe Files (`p`):** Used for inter-process communication (IPC).
  - **Symbolic Link Files (`l`):** Pointer files that reference another file or directory.

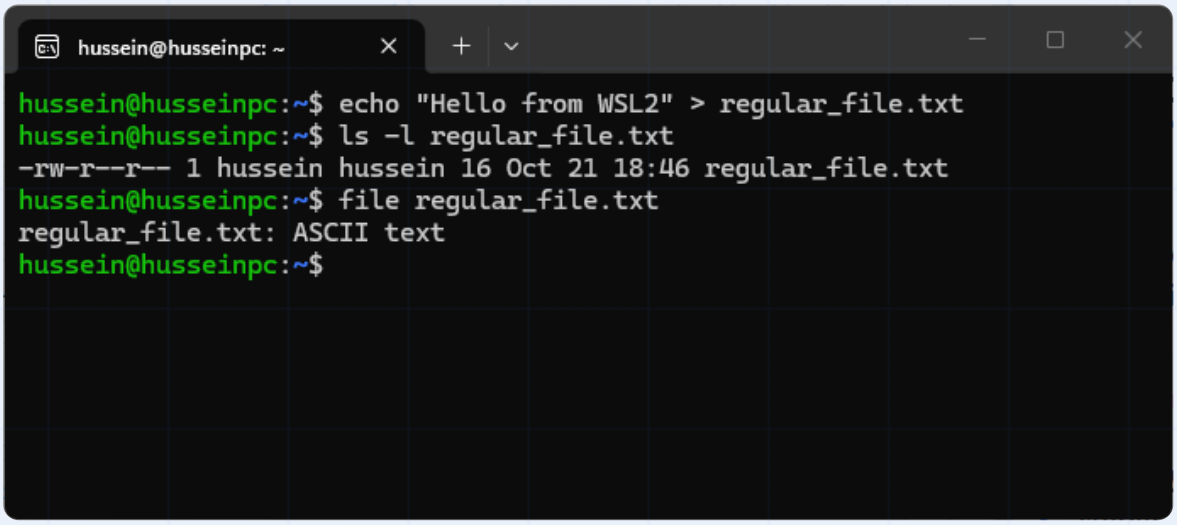
📝 EXAMPLE 2: Regular Files (-)

Regular files are standard data files (like text documents, code, or scripts) represented by a dash (`-`) in the `ls -l` output.

```bash
# Create a regular text file
echo "Hello from WSL2" > regular_file.txt

# Inspect file type and permissions
ls -l regular_file.txt
file regular_file.txt
```

✅ Show Solution



- The leading `-` in `ls -l` indicates it is a **regular file**.
- The `file` command correctly identifies it as text data.

📝 EXAMPLE 3: Directory Files (d)

Directories act as containers holding other files and subdirectories, represented by the letter `d`.

bash

```
# Create a directory
mkdir sample_dir

# Check its file type attributes
ls -ld sample_dir
file sample_dir
```

✅ Show Solution



The leading `d` indicates it is a **directory file**.

3 File Structure

Unlike Windows systems—which store files across separate partition drives such as `C:`, `D:`, and `E:`—Linux organizes all files and directories into a single, unified hierarchical tree structure starting from the **Root Directory (`/`)**.

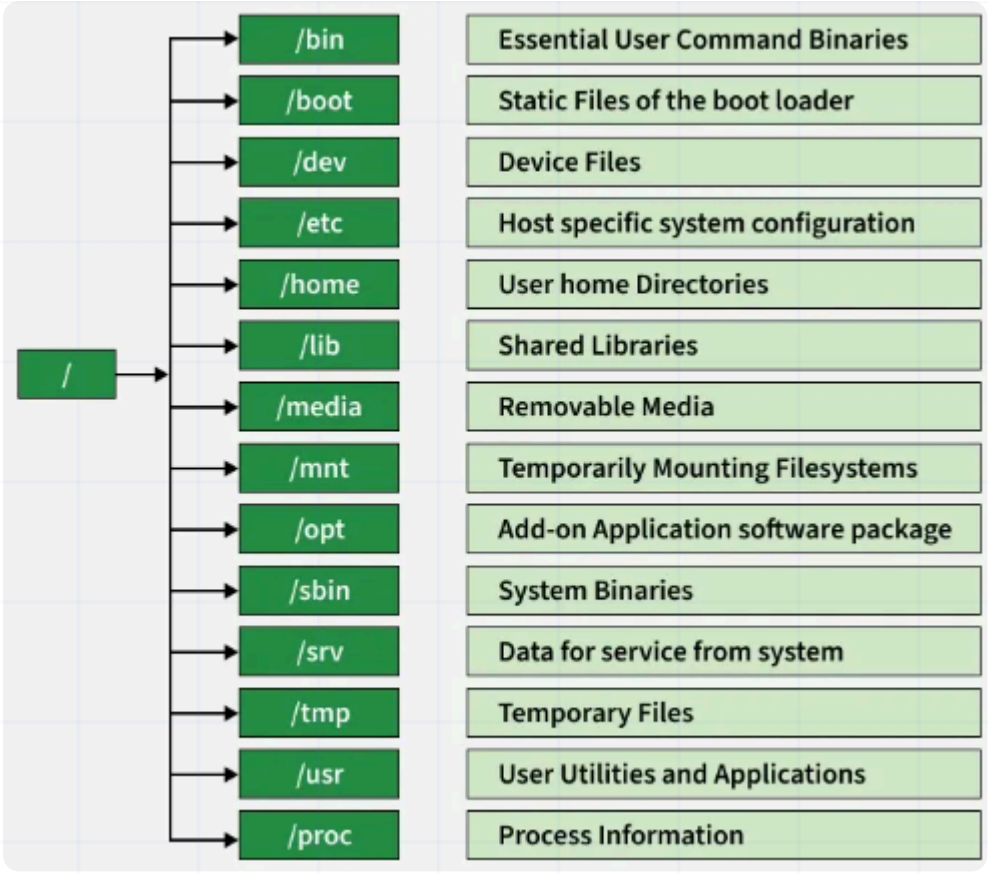


Figure 20: Linux File Hierarchy Structure

Linux relies on a **hierarchical file structure** where every file and folder branches out from a single top-level directory called the **Root Directory**.

- **Root Directory (`/`):** Represented by a forward slash (`/`), it serves as the absolute entry point to the entire file system. Everything on the system—including attached storage devices, mounted partitions, and system configurations—lives under this root directory.
- **User Home Directories (`/home`):** Every user is assigned a dedicated home directory (e.g., `/home/username`) to store personal files, desktop items, and downloads.

3.1 Standard Directory Categories (FHS)

Linux organizes system and user data into specialized standard directories based on their purpose:

- **Binary Directories (البرامج الثنائية):** Contains compiled source code and executable program files.
  - `/bin` : Essential user binaries and common Linux commands available to all users.
  - `/sbin` : System binaries and administrative commands primarily used by the root user.
  - `/lib` : Essential shared libraries required by binaries in `/bin` and `/sbin` .
- **Configuration Directories (مجلدات التكوين):** Contains system and boot configuration files.
  - `/boot` : Contains bootloader and kernel configuration files.*Crucial Note: Modifying or deleting files here can cause the system to fail to boot.*
  - `/etc` : Contains system-wide configuration files (such as network settings, user account data, and application parameters).
- **Data Directories (مجلدات البيانات):** Stores persistent user and service data.
  - `/home` : User home directories.
  - `/tmp` : Temporary storage directory; data here is volatile and can be cleared after system reboots.
  - `/srv` : Contains data for services provided by the system (such as web or FTP data).
- **Memory & Virtual Directories (مجلدات الذاكرة):** Dynamic pseudo-filesystems that do not take up actual hard disk space.
  - `/dev` : Device files used to represent hardware components (e.g., `/dev/sda` for hard drives).
  - `/proc` : Virtual filesystem containing runtime information about active system processes and the kernel.

- `/sys` : Provides detailed information and statistics about kernel configurations and connected hardware.

- **Variable & Shared Directories:**
  - `/var` : Variable directory containing files that constantly change in size (such as logs, cache, and spool files).
  - `/usr` (Unix System Resources): Contains shareable, read-only data, applications, fonts, and themes.
  - `/media` : Default mount point for removable storage media like USB flash drives and CD-ROMs.

3.2 Absolute vs. Relative Paths

When navigating the file system, paths can be defined in two ways:

- **Absolute Path (المسار المطلق):** A complete path starting explicitly from the root directory (`/`) and tracing all subdirectories down to the target (e.g., `/home/hussein/Documents`).
- **Relative Path (المسار النسبي):** A path relative to your current working directory. It never starts with a slash `/` but rather points to locations relative to where you currently stand in the terminal.

> **Note:** Linux file names and paths are **case-sensitive** (e.g., a folder named `Documents` is entirely different from one named `documents`).

> 🎬 SCENARIO 1: **how to navigate between directories using both Absolute Paths and Relative Paths in your Linux terminal (WSL2)**
>
> Let's assume your current working directory is inside your user's home folder: `/home/hussein` , and you want to navigate into a folder named `Documents` located inside `/home/hussein/Desktop/` .

1. Absolute Path (المسار المطلق)

An absolute path **always starts from the Root Directory (`/`)** and traces the exact full path down to the destination, regardless of where you are currently standing in the terminal.

```bash
cd /home/hussein/Desktop/Documents
```

How it works: It explicitly tells the system:"*Start at the absolute root (`/`), go into* `home` *, then* `hussein` *, then* `Desktop` *, and finally into* `Documents` ." You can run this command from **anywhere** on your system, and it will take you straight to the target folder.

2. Relative Path (المسار النسبي)

A relative path **starts from your current location** and navigates forward or backward relative to where you stand right now. It **never** starts with a forward slash (`/`).

A. Navigating Forward into a Subfolder:

If you are currently in `/home/hussein` and want to enter `Desktop/Documents` :

```bash
cd Desktop/Documents
```

- **How it works:** Because it doesn't start with `/` , Linux looks inside your *current* directory (`/home/hussein`), finds the `Desktop` folder, and goes straight into `Documents` .

B. Navigating Using Parent Directory Shortcuts (`..`):

Suppose you are currently deep inside `/home/hussein/Downloads` , and you want to jump over to `/home/hussein/Desktop/Documents` .

```bash
cd ../Desktop/Documents
```

How it works:

- The `..` tells the system to step back **one level up** (from `Downloads` back to `/home/hussein`).
- From there, it immediately steps forward into `Desktop/Documents`.